Embedding Live Machine Learning in Csound: Emotion-Driven Harmony via an ONNX-Runtime Opcode

Anonymous Author(s)

Anonymous Institution
anonymous@example.com

Abstract. Harmonic generation in Csound typically requires explicit chord specification in the score or coordination with an external process. We present the *Affective Harmony Machine* (AHM), a system that embeds an ONNX-format machine-learning model as a native Csound opcode, demonstrated through affect-driven chord progression generation. The system is backed by an annotated dataset of 108 jazz and pop chord progressions labeled with six emotion classes and seven scale/mode categories. A Random Forest classifier trained on this dataset is exported to the ONNX interchange format. The authors created `emoChord`, a Csound opcode that accepts an emotion string, performs live ML inference at i-rate via the ONNX Runtime C API, and schedules harmonically appropriate chord events into a synthesis instrument. No Python interpreter or external process is required at runtime; the current release targets macOS (arm64 natively, x86_64 via Rosetta). The implementation demonstrates a general pattern for embedding ONNX-compatible classifiers in Csound as a native plugin with no Python runtime requirement.

Keywords: Csound opcode, ONNX Runtime, machine learning, harmony generation, emotion, chord progression, affective computing

1 Introduction

Csound [7] has a long tradition of algorithmic composition. John ffitch's `csbeats` pre-processor [9] showed that common-practice notation, including pitches, rhythms, and dynamics, can map directly to Csound score events, and McCurdy's Haiku orchestras demonstrate elegant algorithmic chord generation within Csound. Yet in most of these systems, harmony remains the composer's explicit responsibility: chords are either written into the score by hand or drawn from probability tables constructed in advance. Shi and Boulanger's CStore [1] demonstrated AI-driven orchestra code from natural language, though as an external process, sharpening the question: how far can machine intelligence be embedded *within* Csound itself?

We answer concretely: any ONNX-compatible classifier following the same input/output tensor contract can be embedded as a native Csound i-rate opcode via the ONNX Runtime C API with no external-process dependency. As a demonstration, the *Affective Harmony Machine* (AHM) targets affect-driven harmony, where emotional states map to musical parameters such as mode and perceived affect [2]: a composer writes `i1 0 4 "joyful"` to receive chord events: MIDI-pitched notes inserted at i-rate via Csound's score API (Section 4.2) into any synthesis instrument.

2 Dataset

The system is trained on two curated harmony corpora. The jazz dataset covers functional harmony, including secondary dominants, tritone substitutions, and deceptive resolutions, across 25 unique progressions, each realized in four jazz voicing styles (Four-Way Close, Drop 2, Drop 3, Drop 2+4); the four variants share the same progression class label; voicing style is an auxiliary label for a co-trained classifier, not part of the 108-class target. All progressions and voicing assignments were hand-coded by the authors.[1] The pop dataset covers diatonic, modal, and borrowed-chord idioms across 83 unique progressions. Table 1 summarizes the combined dataset.

Each row is annotated along three orthogonal dimensions: **Emotion** (6 classes: Joyful, Vital, Epic, Uneasiness, Depressive, Despair)[2], **Scale/Mode** (7 classes: Ionian, Aeolian, Dorian, Phrygian, Lydian,

[1] `https://anonymous.4open.science/r/AHM-Dataset-8F24/README.md`

[2] Labels were defined by the authors and intentionally mix grammatical forms (adjective, noun, etc.) as classification shorthand; a principled affect taxonomy is a planned direction (Section 6).

Table 1. Dataset summary after preprocessing.

Source	Progressions	Rows	Coverage
Jazz	25	1,200	12 keys × 4 voicings
Pop	83	996	12 keys
Combined	**108**	**2,196**	–

Mixolydian, Harmonic minor), and **Key** (Key is represented as one of 12 pitch centers rather than as 24 major/minor keys, since mode is encoded separately by the Scale/Mode label). Progressions are encoded in Roman numeral notation (*e.g.,* `I-V-vi-IV`), making the representation key-invariant: the model learns harmonic character, not absolute pitch. Pop progressions were expanded to all 12 keys via automated transposition using key-aware enharmonic spelling tables, ensuring balanced class coverage across all keys. Emotion and scale annotations follow established modal conventions. The six emotion classes were consolidated from thirteen original categories for broader affective coverage.

3 Machine Learning Pipeline

3.1 Feature Design

The classification task is: *given (emotion, scale), predict the most probable chord progression type*, a 2-input, 108-class problem. Key is intentionally excluded from the feature vector because transposition does not alter harmonic character, and including it would fragment the training data into 24 identical shards. Both inputs are label-encoded to integer indices; the feature matrix is shape $[N \times 2]$ (float32), and the target indexes one of 108 progression classes. The dataset spans 23 of the 42 possible `(emotion, scale)` combinations; the model learns a distinct probability distribution over the 108 classes for each observed pair, enabling temperature-weighted sampling rather than a fixed argmax lookup.

3.2 Random Forest with Optuna Tuning

A Random Forest Classifier [3] is chosen because its `predict_proba()` output provides a per-class probability vector over all 108 progression types, directly usable as a stochastic sampling distribution at runtime; the downstream temperature scaling and weighted sampling (Section 4.3) act on this vector to enable probabilistic affective selection.

Hyperparameters are optimized with Optuna [4] (50 trials, 5-fold CV; the training script co-tunes a jazz voicing classifier, but only the progression model is exported). Training follows a two-stage procedure: hyperparameter search on an 80/20 stratified split, then retraining on the complete 2,196-row dataset. Retraining is critical: stratification by emotion (6 classes) rather than the 108-class target means rare progressions absent from the 80% fold would break index mappings at runtime.

3.3 ONNX Export

The trained classifier is exported with `skl2onnx` [5] using `zipmap=False`, which produces a plain float32 tensor `"probabilities"` $[N, 108]$ enabling direct array access from C, and `target_opset=18` to match the bundled ONNX Runtime version. The resulting `gen_model.onnx` encodes the full Random Forest decision structure with no Python dependency. A companion vocabulary table `gen_data.tsv` (108 progressions × 12 pitch classes = 1,296 rows) maps each progression ID to one canonical chord-name string per pitch class; jazz voicing variants share the same progression class and are not represented as separate runtime entries, with major/minor spellings handled according to scale context.

4 Embedding ML in a Csound Opcode

4.1 Overview

The key contribution of AHM is that ML inference runs natively inside the Csound process. The authors built the plugin as a `.dylib` that links directly against the ONNX Runtime C library [6]; `emoChord_init SModelPath, SDataPath` loads the ONNX model and vocabulary table once at i-rate, storing all shared state in static C globals available to every subsequent `emoChord` call (not thread-safe across concurrent Csound instances). Figure 1 shows the two-phase architecture.

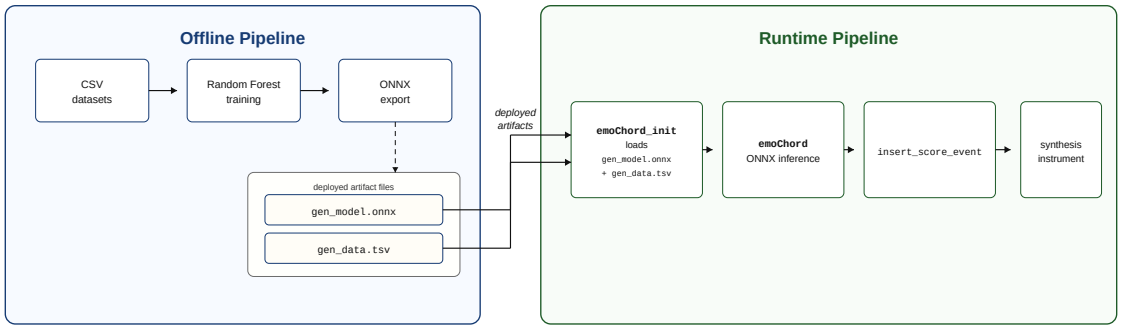

Fig. 1. AHM two-phase architecture. Offline (left): training and ONNX export of `gen_model.onnx` and `gen_data.tsv`. Runtime (right): native Csound inference and chord event scheduling via `emoChord`.

No external process is needed at runtime. Inference runs at i-rate; on Apple Silicon (arm64), median single-sample inference is 14 μs, far below typical score-event scheduling timescales, and model load is 52 ms.

4.2 Live Inference Pipeline in `emoChord`

`emoChord SEmotion, iInstr, iStart, iDur, iAmp [, iTemp, iOctave]` executes the following seven-step pipeline entirely inside the Csound process:

1. **Resolve emotion**: scan `DataEntry[]` for rows matching `SEmotion` (case-insensitive) to obtain `emotion_id`.
2. **Sample scale**: collect all `scale_id` values for that emotion, weighted by dataset frequency; draw one by temperature-weighted inverse-CDF.
3. **ONNX inference**: call `OrtApi->Run()` with input $[1, 2] = \{emotion\_id, scale\_id\}$, retrieving $\mathbf{p} \in \mathbb{R}^{108}$.
4. **Filter**: mask $\mathbf{p}$ to `prog_id` values present for the sampled pair; guaranteed non-empty by construction, since Step 2 draws only from `scale_id` values observed for the given emotion.
5. **Temperature scaling**: apply $p'_i = p_i^{1/T}$ and renormalize over the filtered candidates.
6. **Sample progression**: draw one `prog_id` by weighted inverse-CDF.
7. **Schedule chords**: look up the `chord_name` string (*e.g.,* `Cm7-F7-Bbmaj7`), parse tokens into MIDI note numbers, and call `insert_score_event()` on instrument `iInstr`.

4.3 Temperature as Compositional Control

The `iTemp` parameter (default 1.0) translates the ML temperature concept into a direct musical control. At low T sampling concentrates on the highest-probability progression; at $T = 1$ it follows the model's trained distribution; at $T > 1$ the distribution flattens toward uniform, increasing harmonic variety. A composer can place two adjacent score events with the same emotion string but different temperatures: one to anchor a phrase and one to introduce a surprise chord, without touching the model or any other instrument parameter.

4.4 Usage Example

Listing 1 shows a minimal `.csd`. Instrument 1 reads an emotion string via `strget` and passes it to `emoChord`, which schedules events into any target instrument. Figure 2 shows the console output for the three events in Listing 1.

Listing 1: Minimal `emoChord` usage.

```
<CsInstruments>
emoChord_init "/path/to/gen_model.onnx", "/path/to/gen_data.tsv"
instr 1 ; p4=emotion string
```

```
  Sem strget p4
  emoChord Sem, 2, p2, p3, 0.7
endin
</CsInstruments>
<CsScore>
i1  0  4  "joyful"
i1  6  4  "depressive"
i1 12  4  "uneasiness"
e
</CsScore>
```

Passing the emotion as a score p-field follows the standard Csound idiom [8,9]. At each event, `emoChord` prints the selected progression (*e.g.,* `[emoChord] "joyful" -> Ionian -> C-G-Am-F`), making ML choices transparent during composition.

```
new alloc for instr 1:
emoChord [joyful]: Gm7-C7-Fmaj7
new alloc for instr 2:
new alloc for instr 2:
new alloc for instr 2:
new alloc for instr 2:
  rtevent:      T  1.000 TT  1.000 M:  0.73339
0.73339
  rtevent:      T  2.000 TT  2.000 M:  0.74485
0.74485
B  0.000 ..  6.000 T  6.000 TT  6.000 M:  0.72180
0.72180
emoChord [depressive]: Am-Bm-Em
  rtevent:      T  7.000 TT  7.000 M:  0.57338
0.57338
  rtevent:      T  8.000 TT  8.000 M:  0.57727
0.57727
B  6.000 .. 18.000 T 18.000 TT 18.000 M:  0.57611
0.57611
emoChord [uneasiness]: Ab-Db-Cm-F
  rtevent:      T 19.000 TT 19.000 M:  0.57931
0.57931
  rtevent:      T 20.000 TT 20.000 M:  0.58278
0.58278
```

Fig. 2. `emoChord` console output for three score events.

5 Results and Conclusion[3]

After Optuna tuning (50 trials, 5-fold CV), the model achieves held-out top-1 accuracy of 24.8% and top-3 of 49.1%, against a uniform random baseline of 0.9% (1/108) and a majority-class-per-pair baseline of 24.8%. The latter confirms the model has learned the dominant harmonic preference per `(emotion, scale)` cell; the accuracy ceiling is a design property, since feature vectors within a cell are identical and bounded by the dominant-class proportion. The model's purpose is to package the learned distribution as an ONNX tensor: at $T \to 0$ it reduces to a majority lookup, while temperature sampling exposes the full distribution to the composer.

The dataset built for this work, combining 2,196 rows of jazz and pop progressions under a unified six-class affect taxonomy, is itself a contribution we hope will prove useful to others; we believe this combination to be a first, though the area is broad. Beyond the dataset, the pattern this work establishes is that domain-specific ML is a practical candidate for native Csound integration: a composer writes one emotion word into a score and receives chord events, with all inference invisible inside the process. Any ONNX classifier with a compatible tensor contract can replace the current model without touching the plugin.

6 Future Directions

Planned extensions include a genre input to resolve jazz/pop ambiguity, longer sequence generation, a free-text prompt interface mapping natural-language input to the nearest `(emotion, scale)` pair, a k-rate variant, an `iKey` parameter, emotion label standardization, a perceptual evaluation, and Windows/Linux porting. The authors continue to expand the dataset and refine the model; updated releases will be posted to the project repository as work progresses.

[3] `https://anonymous.4open.science/r/AHM-Dataset-8F24/README.md`

References

1. Shi, L.C., Boulanger, R.: CStore: A Framework for Generating Editable, Versionable Csound Orchestra Code – From Text to Sound and Music. In: Proceedings of the 2026 IEEE Conference on Artificial Intelligence (CAI), pp. 237–243. IEEE, Granada (2026)
2. Hevner, K.: The affective character of the major and minor modes in music. American Journal of Psychology 47, 103–118 (1935)
3. Pedregosa, F. et al.: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)
4. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631 (2019)
5. Dupré, X. et al.: sklearn-onnx: Convert scikit-learn models to ONNX. `https://onnx.ai/sklearn-onnx/` (2019–present)
6. ONNX Runtime Developers: ONNX Runtime inference engine. `https://onnxruntime.ai` (2018–present)
7. Lazzarini, V. et al.: Csound: A Sound and Music Computing System. Springer, Cham (2016)
8. Boulanger, R., Lazzarini, V. (eds.): The Audio Programming Book. MIT Press, Cambridge (2010)
9. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press, Cambridge (2000)